



Imitation Learning for Generalizable Pick-and-Place Operations

Demonstrated on the Emio Platform by Compliance Robotics

Monish Piraimudi, Dan Jung

23rd April 2026

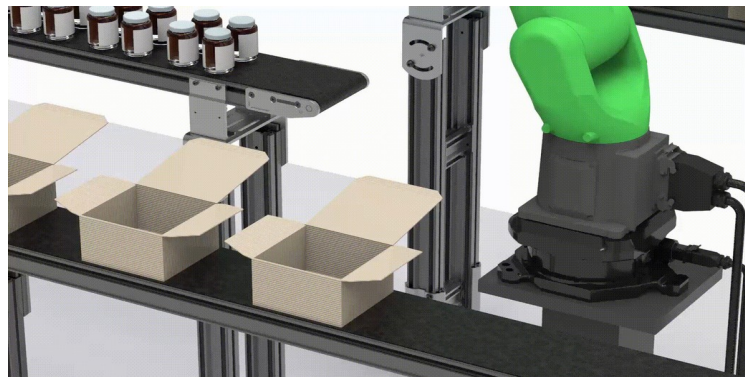
24-786 Advanced Optimization for Engineering – Final Project



Learning to Follow: Automation in Manufacturing

- Modern manufacturing relies on fast, precise, and repeatable pick-and-place operations across industries, from nanofabs to consumer goods.
- Traditional scripted automation works well but requires extensive manual programming and retuning for complex tasks on new product lines.
- As tasks grow more complex, this approach becomes inefficient and difficult to scale.

Hence, generalizable learning methods are desired for their ease of deployment in highly diverse manufacturing environments.



Project Objectives

The objectives of the project are:

1. Implement a complete implicit policy based imitation learning pipeline for the Emio robot.
2. Utilize the learning pipeline to learn a pick and place operation.
3. Demonstrate functionality and sim-to-real transferability by running the trained model on the Emio robot.

Model based Control Methods: PID and MPC Controllers

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

PID Controller:

1. Reactive
2. Linear, low level control

$$\min_{u_{0:T}} \sum_{t=0}^T \ell(x_t, u_t)$$

MPC Controller:

1. Model based predictive controller
2. Requires an explicit system model
3. Convex optimization problem

- PID and MPC rely on predefined models and their accuracy is dependent on the accuracy of modelled system dynamics. High quality modelling of systems requires experts.
- When part geometry, tolerances, or process conditions change, these methods require recalibration, model updates, and constraint retuning.
- As a result, they generalize only within narrow operational envelopes and do not scale efficiently to high variability manufacturing environments.

Learning based Control Methods

$$\pi(u \mid x) \quad \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T r_t \right]$$

$$Q(x, u) = r(x, u) + \gamma \mathbb{E}_{x'} \left[\max_{u'} Q(x', u') \right]$$

Deep Reinforcement Learning:

Find a policy π_{θ} that maximizes the expected total reward accumulated over time across all possible trajectories for a future time horizon of T .

$$\mathcal{D} = \{(x_i, u_i)\}_{i=1}^N \quad f_{\theta}(u) \approx x$$

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(u_i), x_i)$$

Supervised Learning:

Find the model parameters such that the average difference between predictions and true labels across all training data is as small as possible.

DRL is less suitable for manufacturing pick-and-place systems because:

1. It is sample-inefficient
2. Highly sensitive to reward design
3. Requires unsafe exploration for generalization
4. Has slow or unstable convergence under real-world constraints.

Deep Reinforcement Learning Doesn't Work Yet

By Alex Irpan

$$\pi(u | x)$$

$$\max_{\pi}$$

$$Q(x, u) = r(x, u) + \gamma \mathbb{E}_{x'}$$

Deep Reinforcement Learning:

Find a policy π_{θ} that maximizes the reward accumulated over time across trajectories for a future time horizon.

DRL is less suitable for manufacturing

1. It is sample-inefficient
2. Highly sensitive to reward design
3. Requires unsafe exploration for training
4. Has slow or unstable convergence

Sorta Insightful

Reviews

Projects

Puzzles

Archive

Research

About



In a world where everyone has opinions, one man...also has opinions

Deep Reinforcement Learning Doesn't Work Yet

Feb 14, 2018

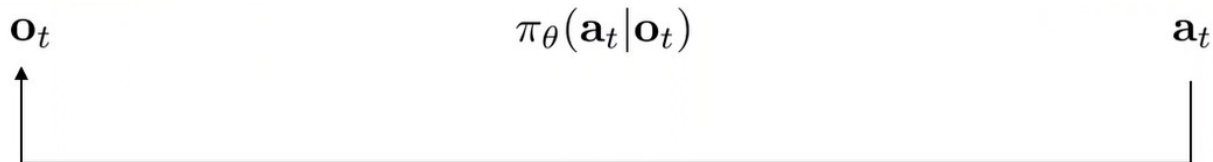
June 24, 2018 note: If you want to cite an example from the post, please cite the paper which that example came from. If you want to cite the post as a whole, you can use the following BibTeX:

```
@misc{r1blogpost,  
  title={Deep Reinforcement Learning Doesn't Work Yet},  
  author={Irpan, Alex},  
  howpublished={\url{https://www.alexirpan.com/2018/02/14/r1-hard.html}},  
  year={2018}  
}
```

This mostly cites papers from Berkeley, Google Brain, DeepMind, and OpenAI from the past few years, because that work is most visible to me. I'm almost certainly missing stuff from older literature and other institutions, and for that I apologize - I'm just one guy, after all.

Supervised Learning of Behaviors: Behavioral Cloning

In behavioral cloning, the learning problem becomes:



$\mathbf{s}_t$ – state

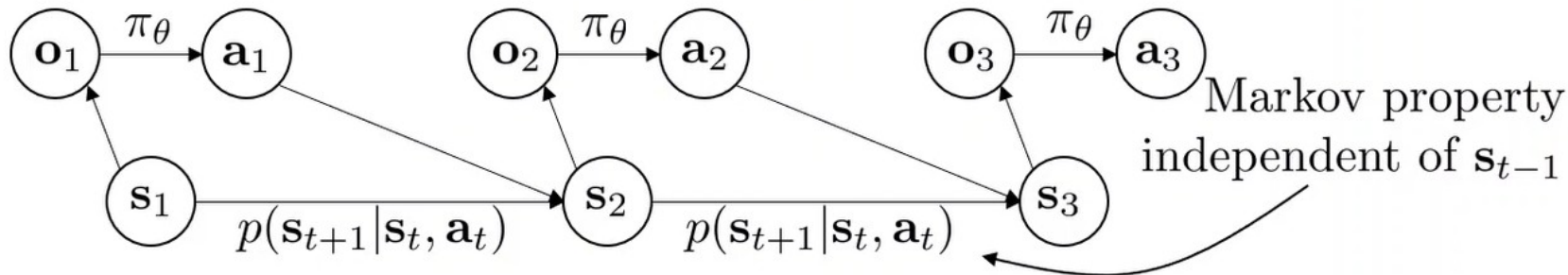
$\mathbf{o}_t$ – observation

$\mathbf{a}_t$ – action

$\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t)$ – policy

$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ – policy (fully observed)

Markov Decision Process:



Towards Generalizable Behavioral Cloning: Implicit Policy

Explicit Policy

Direct Mapping: $\pi_{\theta}(s) = a$

Regression: $\min_{\theta} \mathbb{E}_{(s,a^*)} [\|\pi_{\theta}(s) - a^*\|^2]$

Issues with Explicit Policy:

- Model becomes deterministic or weakly stochastic.
- If multiple valid actions exist: a_1, a_2, a_3 all valid for same s , explicit policy can average them at output, which could be invalid.

Implicit Policy

Minimize Energy: $\pi(s) = \arg \min_a E_{\theta}(s, a)$

Unnormalized conditional distribution over all actions:

$$p_{\theta}(a | s) \propto \exp(-E_{\theta}(s, a))$$

Contrastive NLL Loss on Energy:

$$E_{\theta}(s, a^*) < E_{\theta}(s, a^-) \quad \min_{\theta} - \sum_{i=1}^N \log p_{\theta}(a_i^* | s_i)$$

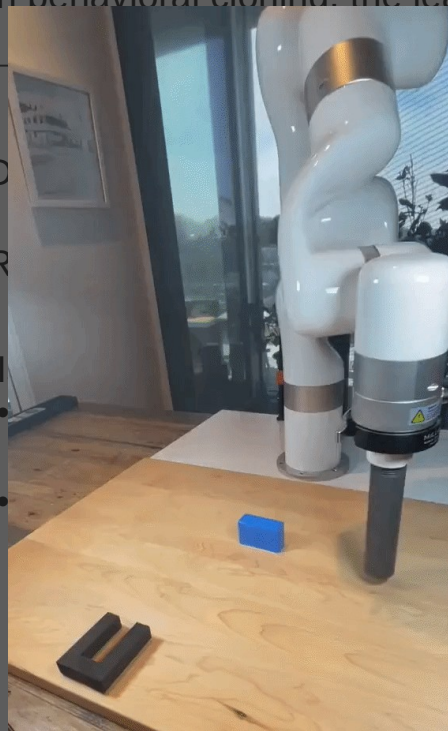
Benefits of Implicit Policy:

1. Learns manifold of valid actions
2. It does optimization during runtime to resolve all possible actions, self updating critic

Implicit Behavioral Cloning

By Florence et al.

In behavioral cloning, the learning problem becomes



Policy

$$\pi_{\theta}(s) = a$$

$$J(\theta) = \mathbb{E}_{(s,a^*)} [\|\pi_{\theta}(s) - a^*\|^2]$$

Deterministic or weakly

may exist: a_1, a_2, a_3 all valid
they can average them to
be valid.

Implicit Behavioral Cloning

Pete Florence, Corey Lynch, Andy Zeng, Oscar Ramirez, Ayzaan Wahid,
Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, Jonathan Tompson

Robotics at Google

Implicit Policy

Minimize Energy:

Unnormalized cost

$$p_{\theta}(a | s)$$

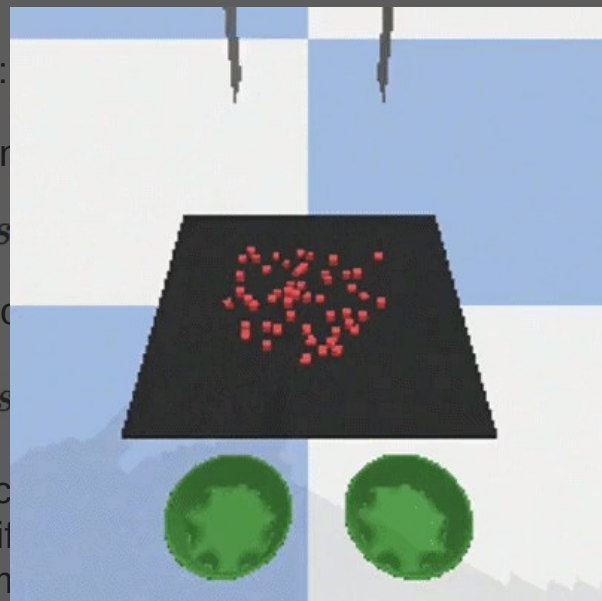
Cross Entropy Loss

$$E_{\theta}(s)$$

Benefits of Implicit

1. Learns manifold
2. It does optimization

possible actions, self updating critic



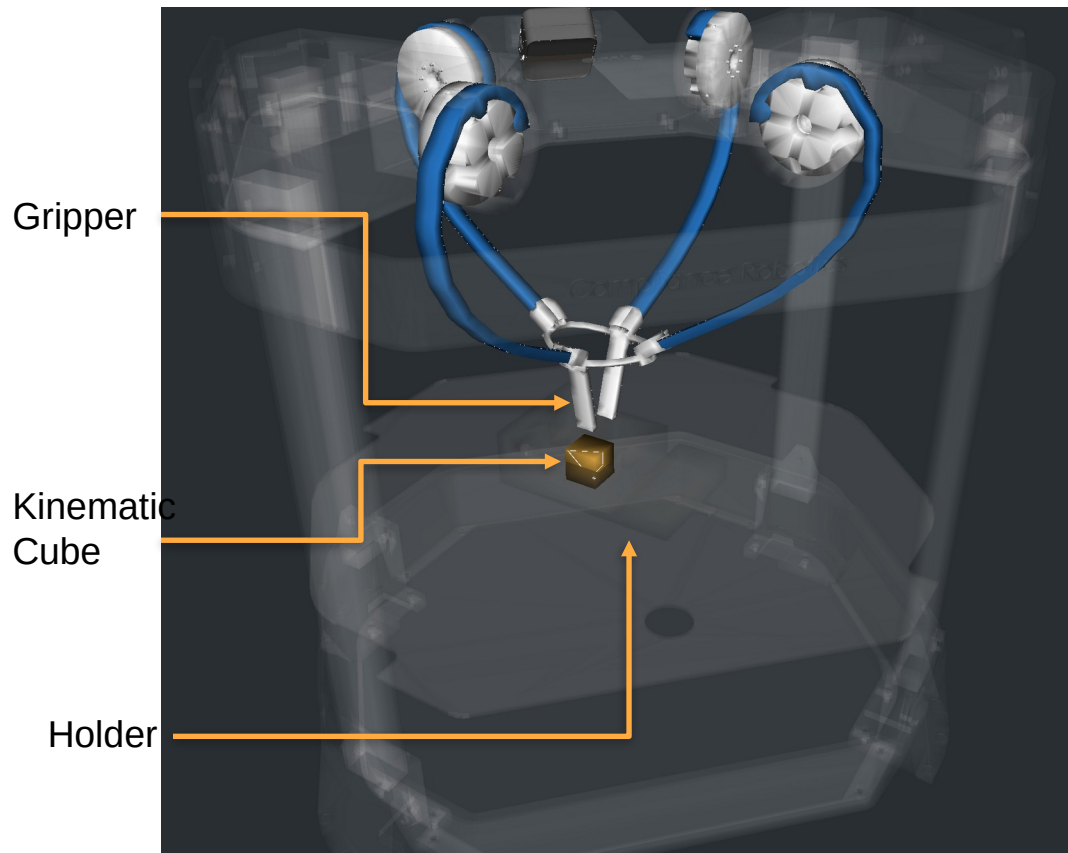
Project Implementation on Emio-Labs and SOFA

Scene Definition and SOFA Environment

We setup our SOFA scene/environment similar to practical 3.

The project is organized around one central scene which is the source of truth for robot and task geometry, expert task execution, etc.

We implemented each stage of the training workflow as separate .py files but they all reuse the central scene file.



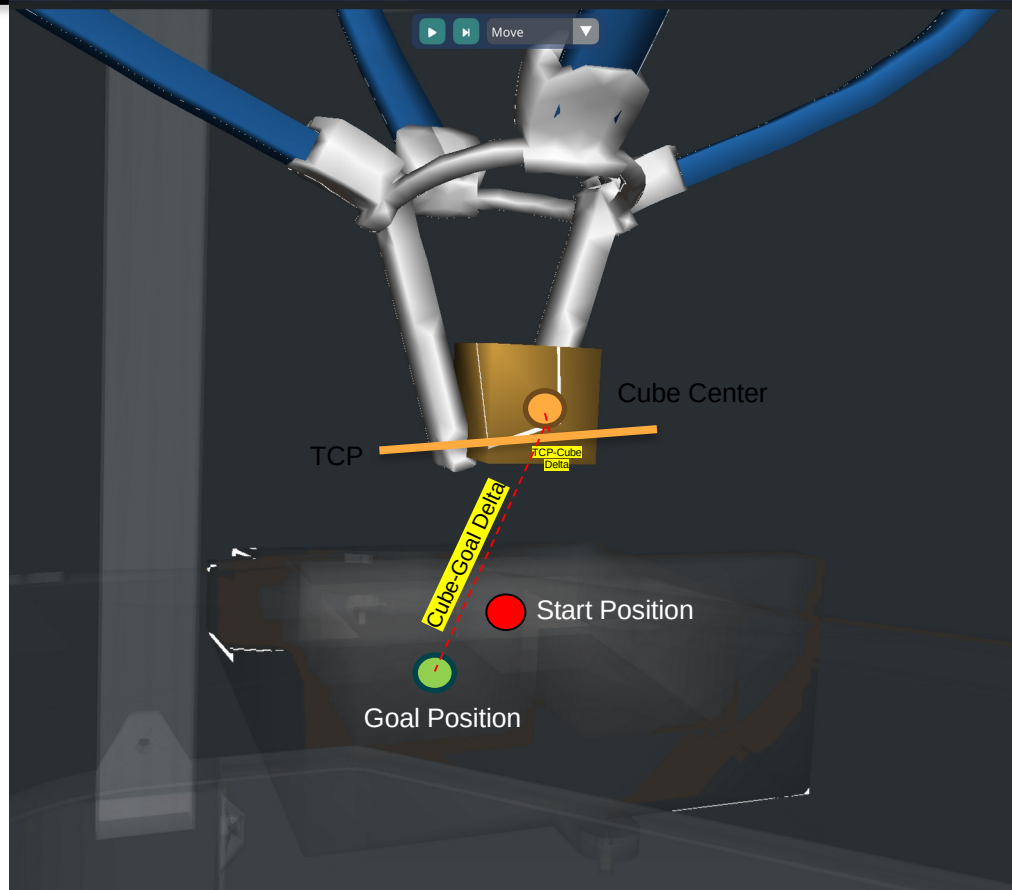
Expert Demonstration Definition and Metrics

The implemented task is a single-cube pick-and-place problem and the demonstrations within the scene are as follows:

1. a cube starts on a tray
2. the robot must approach it, grasp it, lift it, move it to a fixed target, release it, and retreat
3. the cube start position is randomized in a continuous workspace, with default bounds x in $[-35, 10]$ mm and z in $[-30, 20]$ mm

The demonstrations are collected as a dataset of compressed .npz files.

To determine if a gripping action was successful in attaching to the cube, we used a simple threshold-based objective instead of manually modelling the physics interactions .



Discretization of Operations and Complete State Definition

1 Episode

Approach
to Pick

Descend
to Pick

Close
Gripper

Lift

Approach
to Place

Descend
to Deliver

Open
Gripper

Retreat to
Home

The data required to fully define a state at any time point in our simulation is:

Geometric State Vector:

- TCP position (x,y,z)
- cube position (x,y,z)
- goal position (x,y,z)
- TCP-to-cube delta (x,y,z)
- cube-to-goal delta (x,y,z)
- normalized gripper opening
- held flag

Motor Action Vector:

- Motor 1 angle ()
- Motor 2 angle ()
- Motor 3 angle ()
- Motor 4 angle ()

Performance Metrics:

1. Phase index: current task stage
2. Object pose: cube position orientation
3. Target pose: desired placement pose
4. TCP pose: end-effector reference pose
5. Gripper state: open-close holding status
6. Success flags: task completion indicators

Model Definitions

We use PyTorch and Numpy to define the following staged models for the implicit BC energy model:

State Encoder:

17 → **64** → **64**

maps the 17D state into a learned latent state embedding.

Action Encoder:

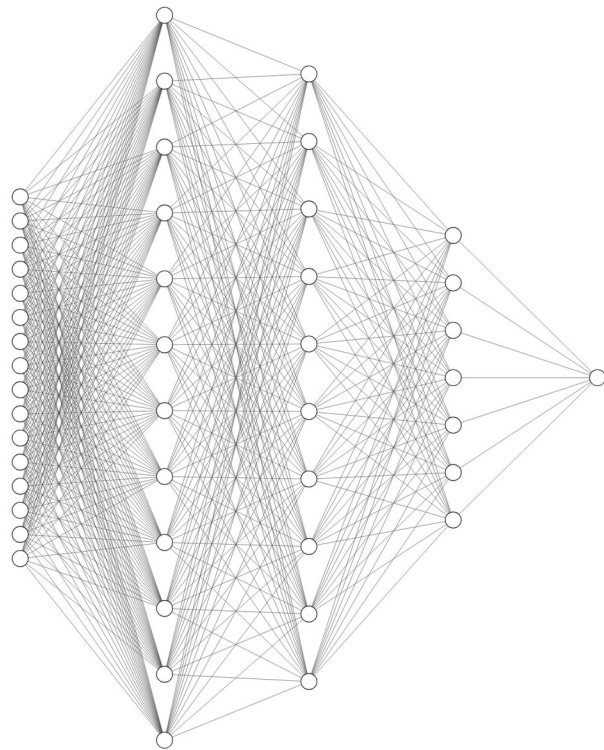
4 → **128** → **128**

maps a candidate 4D motor action into a learned action embedding.

Energy head:

(64 + 128) 192 → **256** → **1**

concatenates both embeddings and outputs one scalar energy $E(s,a)$.



Low level optimization strategy

We are using two different optimization layers separated by training and inference stages in our pipeline:

Optimizer during Training:

Mini-Batch ADAM

We use mini-batch training implemented through PyTorch's DataLoader with shuffling. Model parameters are optimized using the ADAM optimizer.

Optimizer during Inference:

Cross-Entropy Method (Monte Carlo Search)

Since the policy is implicit, it does not output an action directly and requires solving an optimization problem over actions. So, we use CEM, a derivative-free Monte Carlo sampler.

Since the policy defines an energy landscape over actions, and CEM is an efficient way to search that landscape for low-energy (high-quality) actions.

Emio-Labs Module as a Frontend

We structured the training pipeline into step wise lessons. We then implemented these programs as Emio Labs modules accompanied by theoretical explanations of the underlying methods.

The Emio-Labs frontend was convenient way to package our code as well as provide a pedagogical experience to users, helping them walk through all of our underlying optimization choices and methods.

Emio Imitation Learning

Overview

This lab uses the Emio pick-and-place scene as the source of truth for the full imitation-learning pipeline.

In this lab, you will:

- watch the scripted expert in SOFA
- collect successful expert episodes from that same scene
- train an implicit behavior-cloning policy on saved compact state observations
- evaluate the learned policy in closed loop

By the end of the lab, you should understand how one scene supports the full workflow from expert demonstration to imitation-learning evaluation, and why this lab uses an implicit behavior-cloning policy instead of direct action regression.

Install Dependencies

Task Overview

How Implicit BC Works

Step 1: Watch The Expert In SOFA

Step 2: Collect Expert Demonstrations

Step 3: Inspect The Episode Format

Step 4: Train The State-Only Policy

Step 5: Evaluate The Learned Policy

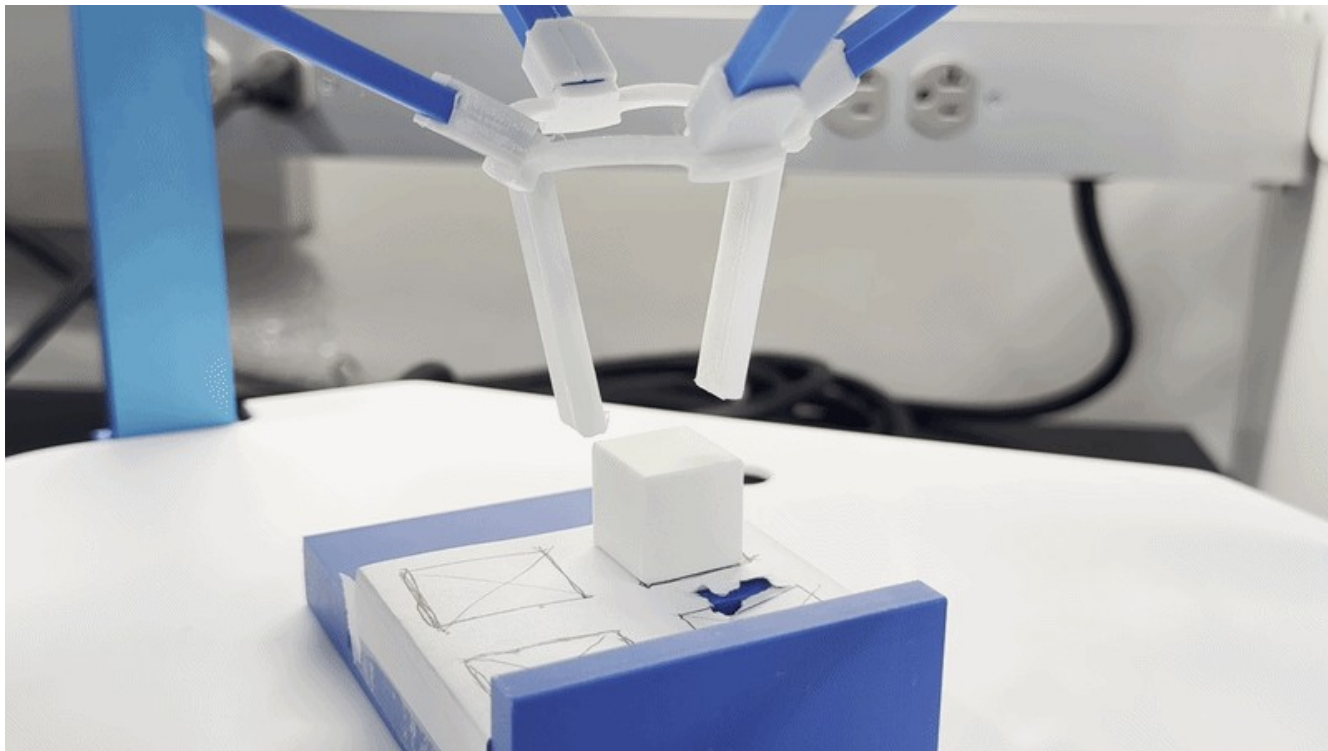
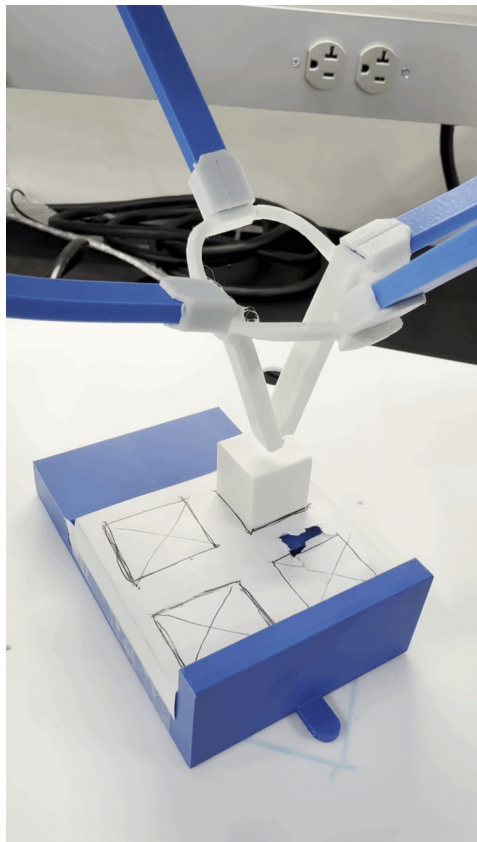
Step 6: Compare Expert And Policy

Summary

Results

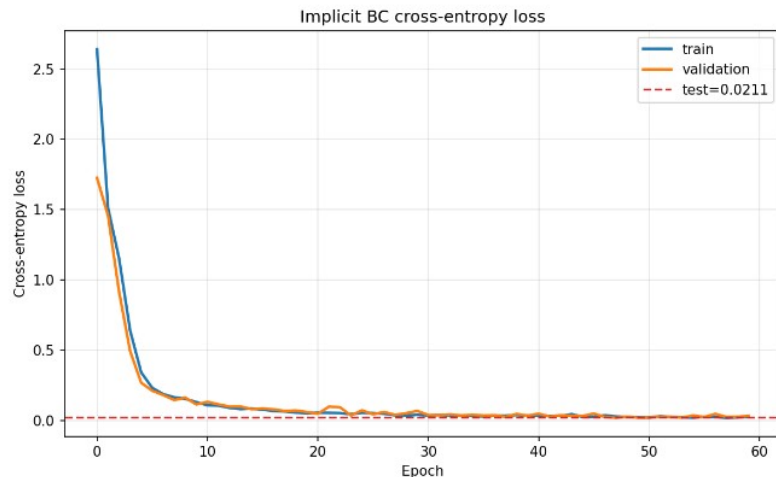
Trained Policy Demonstration within SOFA Environment

Trained Policy Demonstration on Emio Robot



Discussion on Optimization Results

Metric	Value
Saved expert episodes	100
Training / test episode split	80 / 10 / 10
Training / test timesteps	4386 / 557/ 551
Best validation loss	0.0192
Test loss	0.0211
Pick success rate	100%
Place success rate	100%
Total success rate	100%
Median final place error	5.671 mm



Policy successfully learned the expert behavior; close loop performance matched expert levels within few epochs.

- The range of operation was bounded and not diverse

Training and validation cross-entropy losses drop sharply, from roughly the 1.5-2.6 range down to near zero by about 10-15 epochs.

- Model is able to learn the expert state-action structure efficiently
- Plateau after drop is indicative of stable and well-behaved optimization problem

Conclusion

1. We were able to successfully implement a complete implicit imitation learning pipeline for the Emio robot.
2. The learned implicit behavior-cloning policy successfully captures expert's motion, and the training results show strong performance with low validation and test loss.
3. Closed-loop performance matches the scripted expert baseline. The model is able to generalize extensively.
4. Future work could improve robustness by collecting more diverse demonstrations, including failure and recovery cases, expanding to vision-based policy learning, and reducing reliance on scripted sequencing.